MARS-OPS

TEAM DISCUSSION GUIDE

Viva and Project Review Preparation

Architecture | Object Creation | Mission Lifecycle

UML Concepts | Team Questions | Scenarios

Based on the Java source currently available

Missing Mars class, BINDING state, and original UML are identified

MARS-OPS | Team Project Discussion & Viva Preparation Guide

A whole-project study guide

A practical guide to the implemented architecture, object construction, communication, mission lifecycle, and design relationships.

Scope note: This guide describes the source currently available under Prog--MiniProject/src. The project has no class named Mars and includes no original UML diagram or full requirements document. Diagrams here are conceptual diagrams derived from Java references and calls, not reproductions of missing UML. The proposed BINDING state is not implemented.

Contents

1. Scope and project overview
2. Package architecture
3. What happens when a Mars object is created?
4. How objects communicate
5. Mission lifecycle and state machine
6. The single active mission rule
7. UML relationships and evidence
8. Design decisions and OOP
9. Package-by-package guide
10. Team-wide viva questions
11. Deeper module questions
12. Real discussion scenarios
13. Java concepts in this project
14. Final revision sheet

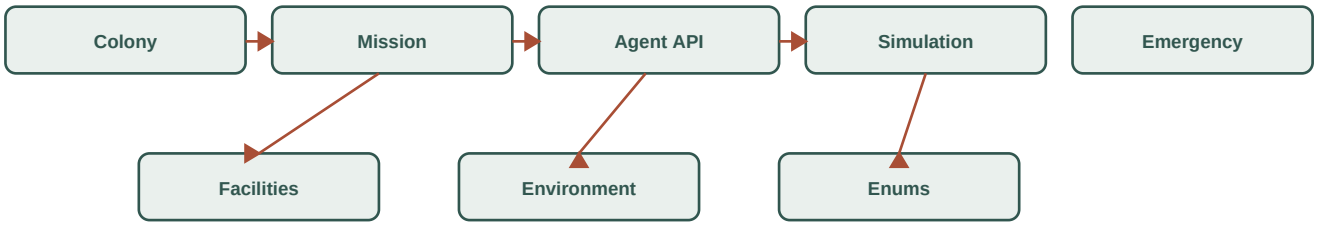
1. Scope and project overview

MARS-OPS is a Java 17 model for managing a Mars colony: personnel, robotic agents, facilities, resources, missions, environmental restrictions, emergencies, and simulated time. The command-line entry point Main.main runs five demonstrations: successful exploration, unsuitable-agent rejection, insufficient resources, oxygen-leak response, and a dust-storm restriction.

The project directory contains a testing playbook PDF with 19 scenarios and 5 walkthroughs. It does not contain the original requirements PDF or UML diagrams. Therefore, all implementation claims below are grounded in Java source; specification and official UML conformity cannot be confirmed.

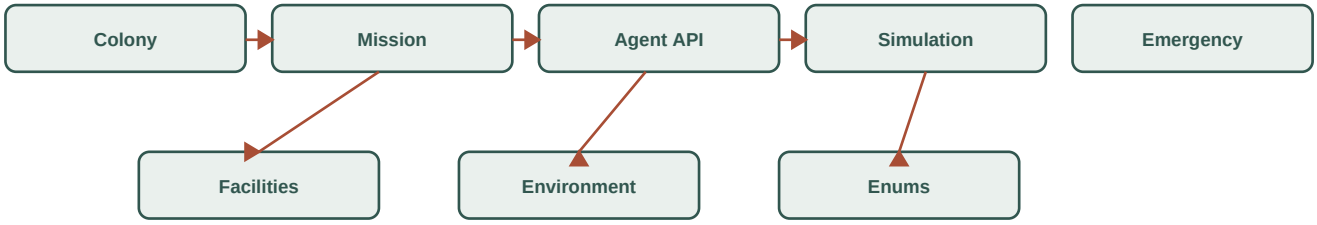
There is no reliable authorship record in the source tree, so this guide does not assign classes to individual team members.

2. Package architecture



Package	Main classes	Responsibility
colony	Colony	Registry and coordinator for agents, facilities, missions, resources, emergencies, and one active mission
missions	Mission, Exploration, Research, Rescue, Repair	Team rules, state, duration, resource requirements, and mission-specific outcomes
agents	Agent	Shared behavioral contract for people and robots
personnel	Personnel, Engineer, Scientist, MedicalOfficer	Human agent data and role-specific mission/emergency eligibility
roboticunit	RoboticUnit, Drone, ExplorationRover, RepairRobot	Robot battery, capabilities, equipment, availability, and mission eligibility
facilities	Facilities, Habitat, Laboratory, StorageFacility, PowerStation, CommunicationStation, Equipment	Facility status, resources, equipment, occupants, and specialized facility behavior
emergencies	Emergency, OxygenLeak, Fire, MedicalEmergency, RoverBreakdown	Occurrence, response requirements, emergency state, and effects
marsenvironment	MarsEnvironment	Mission restrictions and active-mission failure conditions
simulation	Simulation	Static simulation clock and repeated time advancement
enums	Skills, Capability, Resources, and state/type enums	Closed sets of domain values
tests	MissionAssignmentTest	Manual lifecycle and rollback checks

The main execution path is:



Colony coordinates acceptance and execution, but it generally receives already-created objects. Mission owns the team/lifecycle behavior. Agent lets Colony and Mission work with both personnel and robots. Facilities provide the resource stock. MarsEnvironment supplies environmental decisions. Simulation advances the Colony one time unit at a time.

The project has a command-line demo entry point in Main.main. Simulation methods are static; there is no per-Colony Simulation object created by the demo.

3. What happens when a Mars object is created?

The expression Mars mars = new Mars(...) cannot compile against this source: there is no Mars.java or Mars class. The relevant class is MarsEnvironment.

Example actually supported by the project:

MarsEnvironment environment =

new MarsEnvironment(EnviromentStates.NORMAL_CONDITION);

MarsEnvironment's constructor calls SetEnvironmentState. That setter rejects null with IllegalArgumentException and stores the supplied enum in one field. The constructor does not create a Colony, Simulation, Facility, agent, or collection.

Actual object setup in the command-line demo

Main's scenario methods create objects explicitly:

1. A scenario creates a new Colony.
2. It creates a Facility, such as a Habitat, and registers it with addFacility.
3. It creates Personnel and RoboticUnit instances, then registers them.
4. When needed, it creates MarsEnvironment and passes it to Colony.setMarsEnvironment.
5. It constructs a Mission and submits it with a team.
6. Simulation.advanceTime receives the Colony as a method argument when time should move.



Environment constructor stores state only; it creates no Colony

Important distinctions:

• MarsEnvironment is a new object; its constructor only stores a state enum.

• Colony is separately constructed by the caller, not by MarsEnvironment.

• Colony.setMarsEnvironment stores the existing environment reference; it does not clone it.

• Simulation is not constructed. Its static methods receive Colony when called.

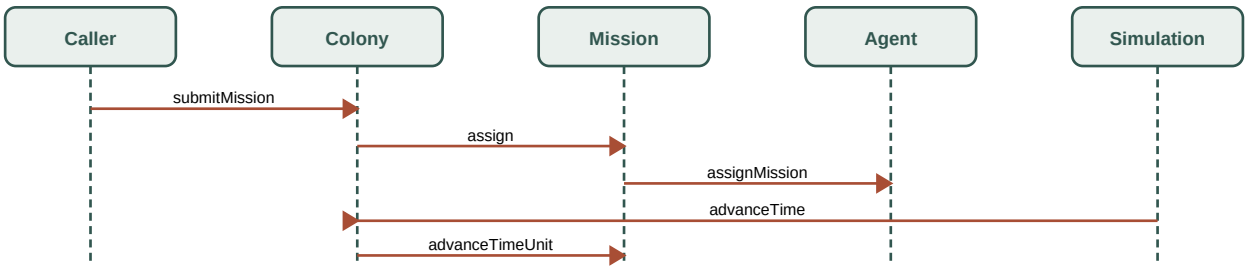
• Facility constructors receive maps/lists and copy the collection containers. The Equipment and Agent objects inside those containers remain shared references.

• Habitat constructs its own LifeSupportSystem inside its constructor.

A useful verbal answer is: "There is no Mars class in this version. MarsEnvironment validates and stores an environment state. The scenario constructs Colony separately and connects the two through setMarsEnvironment."

4. How objects communicate

Communication uses object references, method calls, interfaces, inheritance, and collections:



A typical mission submission proceeds as follows:

1. The caller creates a Mission and a list of Agent references.

2. It calls Colony.submitMission(mission, agents).

3. Colony calls isAvailable and canPerform through Agent. Dynamic dispatch invokes the implementation belonging to the actual Personnel or RoboticUnit object.

4. Colony calls Mission.hasValidTeamComposition, checks MarsEnvironment restrictions when configured, and compares required resources with colony totals.

5. Mission.assign calls agent.assignMission(this). The agent records its mission and becomes unavailable.

6. Simulation.advanceTime(colony, n) calls Colony.advanceTimeUnit repeatedly; Colony advances its active Mission.

7. A terminal transition calls makeAvailable on assigned agents and notifies Colony to clear the active slot.

Constructor-passed references include Mission's facility and requirement collections and RoboticUnit's capability set. Facilities and RoboticUnit keep equipment lists. A defensive list copy protects the list structure, not the Equipment instances themselves. Habitat creates LifeSupportSystem internally.

5. Mission lifecycle and states

Construction and validation

Mission's constructor validates required resources (non-null keys/values, finite nonnegative amounts), required capabilities (non-null set and values), and a positive duration. It copies the maps/sets, creates an empty agent list, stores the facility, and sets state to null. Subclasses add fields such as Exploration.region, Research.topic, and Repair.repairObject. Research specifically requires a Laboratory at construction. Colony does not select a facility automatically; the caller supplies one.

BINDING was discussed but not implemented

The actual enum Mission_Emergency_States contains ASSIGNED, COMPLETED, FAILED, and INPROGRESS. There is no BINDING state and no separate DRAFT enum. A newly constructed Mission has state null, which is used as the pre-assignment draft condition.

Validation is primarily performed in Colony.submitMission. When it rejects through Mission.rejectMission, the mission state becomes FAILED. However, not every unsuccessful submission changes state:

• Empty team, an already-active mission, null/duplicate/unavailable/ineligible agent, invalid team composition, environmental restriction, or insufficient resources calls rejectMission and results in FAILED.

• A null Mission returns false; there is no object whose state can change.

• A Mission already present in Colony.missions returns false without a state change.

• A Mission associated with another Colony returns false without a state change.

• A Mission whose state is already non-null is added to the list and returns false without rejectMission.

Thus "draft becomes ASSIGNED or FAILED" describes the intended common path, but these early-return cases are real exceptions. Do not claim every validation failure becomes FAILED.

Numbered acceptance trace

1. Colony.submitMission checks for null, duplicate registration, association with another Colony, and a pre-existing state.

2. It rejects a null or empty team.

3. It checks whether activeMission is already occupied.

4. For each Agent, it checks null, duplicate membership, availability, canPerform, and prior assignment to this Mission.

5. It calls Mission.hasValidTeamComposition.

6. If an environment is configured, it calls MarsEnvironment.getMissionRestrictionReason.

7. It calls Mission.checkresources using Colony.getColonyResourceTotal.

8. On acceptance, it associates the Mission with the Colony, registers it, and sets activeMission.

9. It calls Mission.assign for each Agent.

10. Mission.assign rechecks the agent, sets assigningAgent as a recursion guard, calls agent.assignMission, appends the Agent, and sets state to ASSIGNED.

11. If assignment throws, Colony rolls back previously assigned agents, clears the slot, and rejects the Mission.

Starting, advancing, and ending

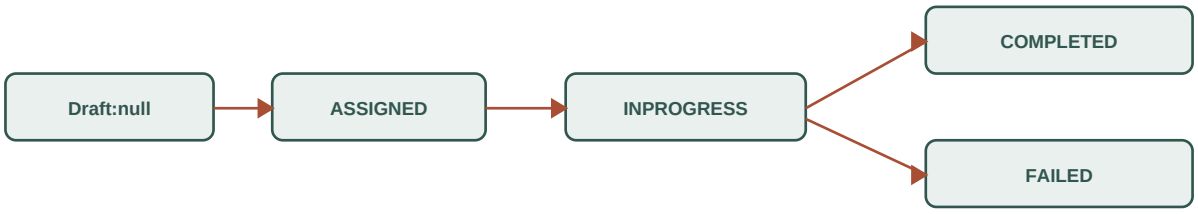
Mission.startMission requires a Colony association, that this Mission is the active one, and state ASSIGNED. It checks agent battery sufficiency, then calls Colony.prepareMissionStart. A battery or resource failure calls failMission. On success, elapsed time resets and state becomes INPROGRESS. That exact spelling is used in the enum; it is not IN_PROGRESS.

Simulation.advanceTime(colony, n) rejects null Colony and n <= 0. For each time unit, it increments the static clock and calls Colony.advanceTimeUnit. Colony checks environmental failure first, then Mission.advanceTimeUnit checks/consumes battery, increments elapsed time, and calls finishMission(true) when elapsed time reaches the required duration. RoboticUnit consumes 1.0 battery per time unit.

Mission resources are deducted once at start by Colony.prepareMissionStart, not once per time unit. Colony first plans allocation across operational facilities, ensures the total is available, snapshots affected inventories, then deducts. If deduction fails, it restores the snapshots. A set tracks missions already prepared so the same mission is not charged twice.

On success, Mission calls onSuccessfulCompletion, changes to COMPLETED, makes every assigned Agent available, clears its team list, and notifies Colony. Failure changes state to FAILED, records a reason, releases agents, clears the list, and notifies Colony.

State machine



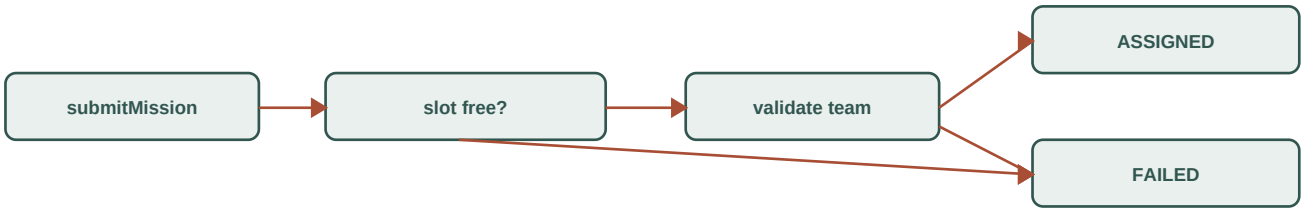
Current state	Operation	Condition	Next state
null (draft)	submit and assign	Validation and assignment succeed	ASSIGNED
null	rejectMission	Nonblank reason	FAILED
ASSIGNED	startMission	Active association, enough battery/resources	INPROGRESS

Current state	Operation	Condition	Next state
ASSIGNED	start with a shortage	Mission calls failMission	FAILED
ASSIGNED	unassign	Remaining team is valid	ASSIGNED
ASSIGNED	unassign last agent	Team becomes empty	null
ASSIGNED	unassign breaks subclass team rule	Mission fails	FAILED
INPROGRESS	advance before duration	Normal progression	INPROGRESS
INPROGRESS	duration reached	finishMission(true)	COMPLETED
INPROGRESS	explicit/environment/battery failure	failMission	FAILED
INPROGRESS	finishMission(false)	Unsuccessful finish	FAILED
Terminal	invalid start/finish/assign	State guards apply	Exception or no-op, depending on method

finishMission before INPROGRESS throws IllegalStateException. advanceTimeUnit outside INPROGRESS is a no-op. Rejection is represented as FAILED; there is no REJECTED state.

6. The single active mission rule

Colony stores one reference, activeMission. In submitMission, the active-slot check is after the null/empty team check and before per-agent validation. A second submission is rejected through rejectMission and added to the missions list; it does not replace or alter the original active mission.



The active reference is cleared through Mission's terminal notification to Colony.clearActiveMissionIfTerminal. Colony.finishActiveMission and Colony.failActiveMission also clear it after calling the Mission. The terminal clear recognizes COMPLETED and FAILED (and null state with an empty team). Assignment rollback clears the slot directly.

"Active" is determined by the activeMission reference. The state enum alone is not the slot. A rejected second Mission becomes FAILED while the first Mission remains active.

7. UML relationships and evidence

No original UML diagrams are present. These are conceptual relationships supported by Java; they are not claims about the missing official drawing.

Relationship	UML notation	Evidence in this project
Association	Plain solid line	Mission stores Colony and Facilities references; Colony stores a single active Mission reference
Generalization / inheritance	Solid line with hollow triangle	Exploration extends Mission; Engineer extends Personnel
Realization	Dashed line with hollow triangle	Personnel and RoboticUnit implement Agent
Dependency	Dashed arrow	Simulation receives Colony as a method parameter

Relationship	UML notation	Evidence in this project
Aggregation	Hollow diamond	Colony stores Agent objects created by callers; plausible interpretation, not provable UML
Composition	Filled diamond	Habitat creates LifeSupportSystem internally; strong ownership evidence, but no UML symbol can be verified

Multiplicity: List<Agent> means the object holds multiple references, but does not by itself prove UML multiplicity or duplicate policy. Colony.activeMission is a single reference. A field or collection supports an association claim; a temporary method parameter supports a dependency claim. A field alone does not prove composition. Habitat's internal creation of LifeSupportSystem supports a composition interpretation.

Point	Aggregation	Composition
Ownership	Weak/shared	Strong/part belongs to whole
Lifecycle	Part can be independent	Part usually follows whole's lifecycle
UML symbol	Hollow diamond	Filled diamond
Cautious example	Colony records externally created Agents	Habitat creates LifeSupportSystem
Evidence limit	Official diagram needed	Java supports interpretation; official UML absent

Generalization and realization are visible in extends/implements declarations. Do not call every reference aggregation or composition. Explain that the relationship type is an interpretation unless the original UML is supplied.

8. Design decisions and OOP

Actual design	Benefit it can provide	Alternative	What can safely be claimed
Agent interface	One behavioral API for different data models	Shared abstract superclass	Structure is certain; author intent is not recorded
Mission subclasses	Common lifecycle with type-specific team rules/outcomes	One class with a mission-type enum and branches	Inheritance is implemented
Colony coordinates submissions	Validation and active-slot policy in one place	Separate application service	This is the current responsibility split
Map<Resources,Double>	One quantity per resource enum	ResourceStock class	Compact aggregation; general facility capacity is not enforced
Sets for skills/capabilities	Unique enum values and membership checks	Lists	Fits set-like domain values
Lists for registries/teams	Preserves insertion order	Sets	Team duplicates are explicitly rejected during submission
Protected hooks	Subclasses customize successful completion or occurrence	Public callbacks	onSuccessfulCompletion and onOccurrence are extension points
Defensive copies	Caller cannot mutate the internal collection structure	Immutable collections	Copies are shallow, so contained objects remain shared

Encapsulation appears in private fields and validation. Polymorphism is used in canPerform, hasValidTeamComposition, applyResponse, and completion hooks. Inheritance appears in the role/robot and mission/facility/emergency hierarchies. Agent is an interface abstraction. Habitat-to-LifeSupportSystem is the strongest composition-like relationship visible in code.

9. Package-by-package study guide

• colony.Colony: registry and coordinator. It generally stores references supplied by callers.

• missions.Mission: small state machine, team, duration, resources, and facility reference.

• agents.Agent: common contract. Its default methods are permissive; a custom Agent may not perform detailed battery/skill checks.

• personnel.Personnel: human data and shared rules. fatigueLevel is stored but not used in canPerform.

• roboticunit.RoboticUnit: battery, capabilities, equipment, and availability.

• facilities.Facilities: resources and status; only operational facilities count in Colony totals. Capacity is not a universal constraint; StorageFacility.store checks it.

• emergencies.Emergency: occurrence and response protocol with subtype behavior.

• marsenvironment.MarsEnvironment: restrictions; it does not own Colony.

• simulation.Simulation: one static clock shared by all colonies in the process, not a clock object per Colony.

• enums: fixed values. Names include EnviromentStates (spelling in source) and INPROGRESS.

• resources.Resources: duplicate resource enum in a different package; domain code uses enums.Resources.

10. Level A: questions every team member should know

1. What is MARS-OPS? A simplified colony, mission, resource, and emergency management model.

2. Is there a Mars class? No.

3. What is MarsEnvironment? Environment state plus mission restriction checks.

4. What does its constructor do? Validates and stores the enum state.

5. Does it create Colony? No.

6. Who creates Colony in the demo? Main scenario methods.

7. How is the environment connected? Colony.setMarsEnvironment stores the reference.

8. Is Simulation constructed? No; it uses static methods.

9. What does Colony coordinate? Registries, mission submission, resources, active mission, emergencies.

10. Why is Agent an interface? It gives people and robots a shared contract.

11. Who implements Agent? Personnel and RoboticUnit.

12. Which canPerform runs? Runtime dispatch selects the actual class override.

13. Where is the active slot? Colony.activeMission.

14. How is a second mission prevented? submitMission checks whether the slot is occupied.

15. When is that check performed? After team null/empty and before each Agent's detailed checks.

16. Does every failed submission set FAILED? No; some early returns leave state unchanged.

17. Why is initial state null? It represents a draft before assignment.

18. Is BINDING implemented? No.

19. Who sets ASSIGNED? Mission.assign after the agent callback succeeds.

20. Who sets FAILED for a recorded rejection? Mission.rejectMission.

21. What prevents assignment recursion? Mission.assigningAgent.

22. How does canPerform differ? Personnel checks role/skill; robots check capability and battery.

23. When are mission resources deducted? At start, not at submission.

24. How does time advance? Simulation → Colony → Mission.

25. What is the exact execution state name? INPROGRESS.

26. When does automatic completion occur? When elapsed units reach the required duration.

27. How are agents released? makeAvailable during terminal transitions.

28. What does a dust storm do? It can block submission or fail an active mission.

29. Does the source show persistent storage? No.

30. Are official UML diagrams available? No; diagrams in this guide are conceptual.

10. Level B: deeper questions by module

Colony/Mission: Why is activeMission a single reference? How does assignment rollback work? How does resource preparation avoid partial deductions? Why does setRequiredResources lock after assignment while some other setters do not? Which early returns skip FAILED?

Agent/Personnel/Robot: What does default hasSkill=false mean? Is fatigue used? What happens when battery is below one unit? How does assigningAgent stop the callback loop?

Facilities/Resources: Are returned collections deep copies? Why does deductResource return the amount actually deducted? Which facilities count toward totals? Is capacity universal? What if the same Equipment instance is stored in two lists?

Emergency/Environment: When is the emergency effect applied? Are resources restored after a response exception? What state remains after RoverBreakdown response? How do pre-submission restrictions differ from active mission failure checks?

Tests: Which four cases does MissionAssignmentTest check? What edge cases are not covered? Why should the test playbook and implementation be checked separately?

For each team member's own contribution, prepare: purpose, inputs, validations, object fields changed, callers, and one realistic edge case. Do not assume authorship from package names.

11. Real discussion scenarios

Starting situation	Actual calls and result
Construct Mars	Does not compile; no class. MarsEnvironment does not construct the rest of the system
Submit while a Mission is active	New Mission is rejected as FAILED and registered; old active Mission remains
Agent unavailable	rejectMission, state FAILED
Agent lacks role/skills/capability	canPerform false, then rejection
Insufficient resources at submission	checkresources false; rejection before deduction
Valid team and resources	Colony association/active slot, then assignment; ASSIGNED
Same Agent appears twice	submitMission rejects duplicate; Mission.assign also rejects an already assigned Agent
Start before assignment	Mission.startMission throws IllegalStateException
Successful timed execution	start deducts resources; advance consumes battery; completion releases team and clears slot
Execution failure	failActiveMission → failMission → FAILED, release, clear slot
Assign after start	INPROGRESS is not an assignable state, so Mission.assign rejects
Finish in invalid state	exception unless current state is INPROGRESS
Facility resources are insufficient	colony totals fail; no deduction
Shortage during preparation	allocation plan fails before deduction; no partial deduction
Facility capacity is exceeded	only StorageFacility.store enforces capacity
Repair succeeds	hook calls performRepair, target becomes OPERATIONAL, Mission becomes COMPLETED

12. Focused Java reference

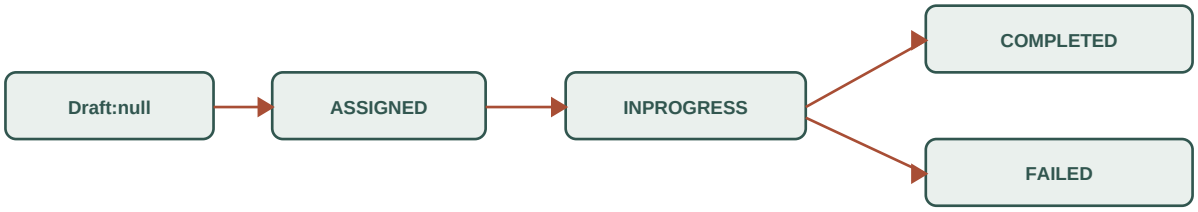
Concept	MARS-OPS example
private/protected/public/package access	Mission fields are private; hooks protected; Package-private access is limited to classes in the same package
extends / implements	Exploration extends Mission; Personnel implements Agent
@Override	Engineer and Scientist override canPerform
Abstract class / interface	Mission shares lifecycle state; Agent defines a behavioral contract
Constructor overloading	Emergency has multiple constructor signatures
Overload / override	MedicalOfficer overloads canPerform; Scientist overrides it
this / super	Current object / parent constructor or behavior
instanceof	Repair checks Equipment or Facilities target
Generics	List<Agent> constrains list element types
ArrayList / HashSet / HashMap	registries / skills / resource quantities
throw and validation	Invalid duration or NaN resource amount raises an exception
Defensive copy	Getter returns a new list but the elements are the same references
null and state	null represents draft; transition guards check state
Polymorphism	agent.canPerform calls the runtime class implementation

13. Final revision sheet

A. The project in one minute

Colony coordinates Agents, Facilities, Missions, resources, and emergencies. Agent unifies people and robots. submitMission validates the team, environment, and resources; start deducts resources; Simulation advances time; Mission completes or fails and releases its team. One activeMission reference prevents a second accepted mission. MarsEnvironment is separate and does not create Colony.

B. Lifecycle cheat sheet



Draft null → validation → ASSIGNED → start/resource deduction → INPROGRESS → COMPLETED or FAILED → release agents and clear active slot. Recorded rejection becomes FAILED, with early-return exceptions.

C. UML quick reference

Association: plain line. Generalization: solid line with hollow triangle. Realization: dashed line with hollow triangle. Dependency: dashed arrow. Aggregation: hollow diamond. Composition: filled diamond. Examples above are interpretations from source; the original UML is unavailable.

D. Top 20 likely questions

1. Is there a Mars class? No.

2. Does MarsEnvironment create Colony? No.

3. Is BINDING implemented? No.

4. What does null state mean? Draft.

5. What is the execution-state spelling? INPROGRESS.

6. Where is the active slot? Colony.

7. Does rejection of a second mission replace the first? No.

8. Does every failed submission change state? No.

9. Who sets ASSIGNED? Mission.assign.

10. When are resources deducted? At start.

11. How does preparation avoid partial deduction? Plan, check, snapshot, rollback.

12. Who advances time? Simulation.

13. Battery usage per robot per time unit? 1.0.

14. What happens on successful completion? COMPLETED and release.

15. What happens on failMission? FAILED, reason, release.

16. Why Agent interface? Shared contract.

17. Example of dynamic dispatch? Agent.canPerform.

18. Is data persisted? No evidence of persistence.

19. Are project UML diagrams available? No.

20. Are guide diagrams official UML? No, they are conceptual.

E. Final team checklist

• [ ] Explain that Mars class is absent.

• [ ] Distinguish MarsEnvironment from Colony and Simulation.

• [ ] Draw submission, active slot, start, and finish flow.

• [ ] Use INPROGRESS exactly as named in source.

• [ ] Do not claim every rejection becomes FAILED.

• [ ] Explain assigningAgent and rollback.

• [ ] Distinguish resource checks from deduction.

• [ ] Explain agent release and active-slot clearing.

• [ ] State that official UML is unavailable.

• [ ] Do not present BINDING as implemented.

• [ ] Explain one class from every domain package.

• [ ] Mention test limits and lack of persistence evidence.